Centroid Text Classification

1. Import Required Libraries

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np
```

Purpose

TfidfVectorizer

- Converts text into TF-IDF feature vectors.

cosine_similarity

- Measures how similar two vectors are.

NumPy

- Used for numerical operations and array conversion.

2. Training Documents

```
documents = [
    "machine learning and artificial intelligence",
    "deep learning neural networks",
    "natural language processing",
    "football world cup tournament",
    "cricket match highlights",
    "soccer championship players"
]
```

The dataset contains two conceptual groups:

Technology Documents

```
machine learning and artificial intelligence
deep learning neural networks
natural language processing
```

Sports Documents

```
football world cup tournament
cricket match highlights
soccer championship players
```

These documents are used to build category centroids.

3. Category Labels

```
labels = [
    "Technology",
    "Technology",
    "Technology",
    "Sports",
    "Sports",
    "Sports"
]
```

Each document is assigned a category.

Document Group	Category
AI, ML, NLP Documents	Technology
Football, Cricket, Soccer Documents	Sports

4. TF-IDF Vectorization

```
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(documents)
```

What Happens?

The vectorizer:

1. Learns vocabulary from all documents.
2. Computes TF-IDF weights.
3. Converts each document into a numeric vector.

Example:

```
"deep learning neural networks"
```

↓

```
[0.12, 0.33, 0.56, ...]
```

Each value represents the importance of a term.

5. Find Unique Categories

```
categories = list(set(labels))
```

Result:

```
["Technology", "Sports"]
```

These categories will each receive a centroid.

---

6. Create Centroid Dictionary

```
centroids = {}
```

Structure:

```
{
    "Technology": centroid_vector,
    "Sports": centroid_vector
}
```

7. Compute Category Centroids

```
for category in categories:
```

The program processes one category at a time.

Get Category Documents

```
indices = [
    i for i in range(len(labels))
    if labels[i] == category
]
```

For Technology:

```
[0, 1, 2]
```

For Sports:

```
[3, 4, 5]
```

8. Important Improvement

```
centroid = np.asarray(
    X[indices].mean(axis=0)
)
```

Why This Change?

In the previous version:

```
centroid = X[indices].mean(axis=0)
```

returned a **NumPy Matrix**.

Some versions of Scikit-learn may raise:

```
TypeError:
np.matrix is not supported
```

when used with:

```
cosine_similarity()
```

To avoid this issue:

```
np.asarray(...)
```

converts the matrix into a standard NumPy array.

Example

Before:

```
matrix([[0.1, 0.2, 0.3]])
```

After:

```
array([[0.1, 0.2, 0.3]])
```

This makes the centroid compatible with newer Scikit-learn functions.

9. Store the Centroid

```
centroids[category] = centroid
```

Example:

```
{
    "Technology": [[0.15, 0.33, 0.42,...]],
```

```
    "Sports": [[0.52, 0.07, 0.01,...]]
}
```

10. Test Document

```
test_doc = [
    "neural network and machine learning"
]
```

This unseen document needs classification.

```
neural
network
machine
learning
```

which are Technology-related.

---

11. Convert Test Document

```
test_vector = vectorizer.transform(test_doc)
```

The TF-IDF model learned during training transforms the test document into a numeric vector.

12. Initialize Variables

```
best_category = None
highest_similarity = -1
```

Purpose:

```
best_category
```

stores the final predicted class.

```
highest_similarity
```

stores the largest similarity value found so far.

13. Compare with Each Centroid

```
for category in categories:
```

The test document is compared with every category centroid

Calculate Cosine Similarity

```
similarity = cosine_similarity(
    test_vector,
    centroids[category]
)[0][0]
```

Formula

Where:

- A = Test document vector
- B = Category centroid

Interpretation

Similarity Value	Meaning
1	Perfect match
0	No similarity
Close to 1	Highly similar

14. Display Similarity Scores

```
print(
    category,
    "Similarity:",
    round(similarity, 4)
)
```

Possible Output:

```
Technology Similarity: 0.7412
Sports Similarity: 0.0785
```

15. Select the Best Category

```
if similarity > highest_similarity:
```

The category with the highest similarity is chosen.

```
highest_similarity = similarity
best_category = category
```

16. Display Final Prediction

```
print(
    "\nPredicted Category:",
    best_category
)
```

Expected Output:

```
Technology Similarity: 0.7412
Sports Similarity: 0.0785

Predicted Category: Technology
```

because the test document is much closer to the Technology centroid.

Execution Flow

```
Training Documents
        ↓
TF-IDF Vectorization
        ↓
Compute Category Centroids
        ↓
Convert Centroids to NumPy Arrays
        ↓
New Test Document
         ↓
 TF-IDF Conversion
```

```
        ↓
Cosine Similarity with Each Centroid
        ↓
Choose Highest Similarity
        ↓
Predicted Category
```

Difference Between Version 1 and Version 2

Feature	Version 1	Version 2
TF-IDF	✅	✅
Category Centroids	✅	✅
Cosine Similarity	✅	✅
NumPy Array Conversion	❌	✅
Avoids np.matrix Errors	❌	✅
Scikit-learn Compatibility	Limited	Better

Key Takeaway

centroid_text_classification2.py implements a **Centroid-Based Text Classifier** using **TF-IDF** and **Cosine Similarity**. The main enhancement over the earlier version is:

```
centroid = np.asarray(
    X[indices].mean(axis=0)
)
```

This converts centroid vectors from a NumPy matrix to a standard NumPy array, preventing potential TypeError issues when computing cosine similarity and improving compatibility with modern Scikit-learn versions.